METROINSIGHTS / INTERVIEW STUDY SERIES

Master Project Overview

A repository-grounded guide to the product, architecture, data pipeline, model, service simulation, network graph, and interview story.

PHASE 1 | VERSION 1.0 | 14 OCTOBER 2026

THE PROJECT IN ONE PARAGRAPH

MetroInsights is a Flask dashboard that lets a user explore simulated Delhi Metro station demand and service-planning scenarios. It combines a saved scikit-learn forecaster trained on generated data, a transparent rule-based headway calculator, estimated line shares at interchanges, and a topology-aware shortest-time graph. It is a software and simulation prototype; it is not connected to live passenger counts, train positions, or verified current operating schedules.

INTERVIEW INTEGRITY

The model metrics describe performance on this project's synthetic holdout. They are not observed DMRC forecasting accuracy. Capacities, line shares, and schedule limits are configurable project assumptions until verified against authoritative operational sources.

228	3.15M	23	11
station records	generated rows	model features	project line/branch sequences

Built from the tracked MetroInsights repository and its project data audit. This guide explains what the code does, why the components matter, what the numbers mean, and where the evidence ends.

1. Product purpose and scope

The problem it demonstrates

Transit planners often need to compare demand patterns with service frequency. MetroInsights demonstrates a workflow for selecting a station and scenario, estimating demand, comparing it with an assumed boarding-capacity budget, and seeing how a rule-based service recommendation changes. The system makes the assumptions visible enough to discuss and improve.

Who it serves

- **Implemented user:** a dashboard user exploring station, date, time, and line scenarios.
- **Likely audience (inference):** reviewers, students, or planning stakeholders evaluating a prototype. The repository has no authentication, role model, or operational control permissions.

What it accepts and returns

Input	Output
Station, date, time, optional line	Selected-hour simulated demand and minute-interpolated estimate
Date-derived weekday / listed occasion	Daily operating-hours curve and peak hour
Station line metadata and scenario assumptions	Suggested headway, trains/hour, estimated clearance utilization, wait estimate, explanation
Origin station topology	Shortest estimated travel time to reachable destination stations

SCOPE BOUNDARY

The project does not know the passenger load already aboard a particular train, how many people will exit at the selected station, or the number who will board that train. Its utilization KPI compares estimated station-hour demand with a modeled per-hour boarding-clearance capacity.

Core workflow

1. Select	2. Forecast	3. Allocate	4. Simulate	5. Explore
Station + scenario	Saved ML model	Synthetic line shares	Headway rules	Charts + routes

The landing page is at `/`; the interactive workspace is `/dashboard`. The main page uses JavaScript to call `/api/predict`. `/get_footfall` remains as a synchronous form/report route.

2. Architecture and execution flow

The application is a single Flask web process with a browser UI. It reads local CSV/JSON configuration and prebuilt model artifacts. There is no database, queue, external demand API, or persistent user-scenario store.

Runtime architecture

Browser HTML + vanilla JS	**Flask routes** Validate + orchestrate	**Forecast module** joblib model	**Schedule rules** CSV assumptions
Travel graph: topology + coordinates + Dijkstra returns estimated destination times			
Response: JSON drives KPIs, line allocation table, and Chart.js visualizations			

Representative request, end to end

- **1.** `dashboard.js` reads the station/date/time/line controls and POSTs JSON to `/api/predict`.
- **2.** `routes.py` parses the date and time, derives calendar context, and checks the modeled 05:00-23:30 service window.
- **3.** `forecasting.py` assembles a 23-feature row, predicts the hourly estimate, interpolates the selected minute, and builds the 05:00-23:00 curve.
- **4.** `scheduling_optimizer.py` allocates interchange demand by configured shares and computes a service scenario.
- **5.** `travel_graph.py` runs Dijkstra from each line-node at the origin station and chooses the shortest result per destination.
- **6.** The response returns to JavaScript, which updates KPIs, charts, tables, and the journey-time list.

Application startup

`run.py` imports the Flask app. `app/__init__.py` creates it and imports routes. Importing routes reads the cleaned station directory and builds the graph. Importing forecasting loads the serialized estimator and metadata. The production `Procfile` uses `gunicorn run:app`; the local runner reads `PORT` and `FLASK_DEBUG`. This import-time loading avoids per-request artifact loading, but each worker holds its own model and graph in memory.

Key files: `run.py`; `app/__init__.py`; `app/routes.py`; `app/forecasting.py`; `app/static/js/dashboard.js`; `app/templates/index.html`.

3. Data lifecycle and model pipeline

Raw files	Pipeline 01	Pipeline 02	Pipeline 03	Artifacts
Station / line CSVs	Clean + enrich	Generate scenarios	Train + evaluate	Model + metadata

Stage	What the code does	Important output
01 - Audit and clean	Trims station-name markers; patches a small explicit coordinate list; adds line/interchange metadata; writes operations assumptions.	`stations_cleaned.csv`, `metro_line_operations.csv`
02 - Generate	Assigns station typologies/weights; applies day, hour, month, event, and station-type factors; draws Poisson counts with seed 42.	3,153,696 synthetic station-hour rows; line-share JSON
03 - Train	Builds date/time/station features and historical aggregates; splits dates chronologically; compares baselines and trains boosting model.	model `.joblib`, metadata `.joblib`, metrics JSON

Where the generated demand comes from

The two-year scenario spans 2025-01-06 through 2027-01-03: 728 dates, 228 stations, and 19 hourly buckets each day. The generator authors a weekday system target of about 3.2 million daily entries, then distributes that target using line weights, interchange multipliers, and named-hub multipliers. It reshapes demand by weekday/weekend curves, station type, selected holidays/events, and monthly factors. A seeded Poisson draw produces integer counts. No observed station-hour count file is used.

LARGE CSV AND DEPLOYMENT

The generated CSV is about 130 MiB in the local checkout and is ignored by Git. The web app serves forecasts from the committed model and metadata; runtime inference does not load that large CSV. Recreating or retraining the model does require the synthetic CSV, which Pipeline 02 regenerates. The generator also writes a legacy duplicate.

Data categories to keep separate

Category	Examples	How to describe it
Raw project input	Station name, line, coordinates	Static source file; source/date/license still need verification.
Derived metadata	Cleaned stations, operation assumptions, typologies	Project transformations and assumptions, not automatically authoritative.
Synthetic targets	Station/hour counts and interchange shares	Simulated scenarios; not observed ridership or transfer counts.
Model artifacts	Estimator, feature metadata, metrics	Ready for app inference; scores remain tied to synthetic holdout.

4. Forecasting model and evaluation

The model estimates hourly station footfall from structured context. Its inputs combine calendar and cyclical time signals, station identity and type, line/interchange structure, and historical averages. The inference code reconstructs the feature row in the saved metadata column order before calling the model.

Feature families (23 total)

Family	Features / intuition
Calendar	Hour, weekday, month, day of month/year; weekend/workday/holiday/event type.
Cyclical time	Sine/cosine of hour, day of week, and month make the cycle wrap smoothly (e.g., Sunday near Monday).
Station structure	Station ID, line ID, typology ID, interchange flag, number of lines.
Historical context	Station mean, station + weekday + hour mean, station + month mean; gives the model a prior pattern.

Training and validation

- Training data is sorted by date and split before 2026-10-01 versus on/after that date. The committed test window runs through 2027-01-03.
- Training historical means exclude the row being predicted and earlier feature values are used; the holdout receives aggregates computed from training dates only.
- The comparison includes station overall mean, historical station/day/hour mean, Ridge regression, and `HistGradientBoostingRegressor`.
- Model settings in source: 250 iterations, learning rate 0.06, max depth 9, minimum leaf 25, L2 0.05, random state 42.

Synthetic holdout metric	Boosting model	What it means here
MAE	27.90 riders/hour	Mean absolute error on generated targets.
RMSE	54.82 riders/hour	Penalizes larger synthetic residuals more.
R-squared	0.9947	Variance explained in this generated holdout.
MAPE	5.16%	Computed only for non-zero generated targets.

HOW TO DEFEND THE HIGH SCORE

Say: "The chronological synthetic holdout score shows that the model reproduces patterns in our generated scenario data. Because the train and test targets come from the same generator, this score does not establish accuracy on real passengers. The next validation step is a held-out period of observed station-level counts."

Runtime inference

`predict_hourly_footfall()` creates one feature row and clamps negative predictions to zero. `predict_footfall_with_minute_interpolation()` linearly blends the current and next hourly values. `predict_station_full_day_profile()` evaluates hours 05 through 23, identifies the maximum point, sums the 19 predictions, and creates line-specific curves using the interchange JSON. The 23:00 chart point is a full hourly bucket even though selected-time service closes at 23:30; the curve does not resolve the final half hour.

Source map: `pipeline/02_generate_calibrated_dataset.py:239-...`; `pipeline/03_train_and_evaluate_forecaster.py:48-194, 197-...`; `app/forecasting.py:56-188`; `models/evaluation_metrics.json`.

5. Service-planning rules and what utilization means

The schedule module is deterministic business logic over configurable CSV values. It is not another ML model and does not solve a network-wide timetable optimization problem.

Simplified calculation

Step	Logic
Frequency limits	Fleet limit = floor(fleet size x 60 / turnaround minutes). Signaling limit = floor(60 / minimum headway). The stricter limit bounds service frequency.
Baseline	Uses configured peak/off-peak headway, with a taper adjustment at 05:00, 06:00, 22:00, and 23:00.
Boarding budget	Comfort capacity x (1 - assumed through-load share). Through-load is 65% at peak and 40% off-peak; budget has a 250-rider floor.
Recommendation	Compares station/line demand to baseline clearance capacity; selects increase, maintain, or decrease; clamps and rounds to practical headways.
KPIs	Utilization = estimated hourly demand / recommended boarding-clearance capacity. Average wait = headway / 2. Deployed train sets = ceil(turnaround / headway), capped by fleet.

How to explain capacity utilization

Current definition: an hourly scenario ratio. It asks how large the estimated boarding demand is relative to the modeled comfortable boarding-clearance capacity provided by trains per hour. It is not the percentage of seats occupied, the load of a specific train, or a live station crowd measurement.

To model the passenger load on an arriving train: begin with the load entering the segment, subtract expected exits at each upstream stop, then add boardings at each stop. At the selected station, utilization would compare the predicted onboard load after boarding with a verified comfort/crush capacity. That requires direction, segment, stop-level OD/boarding/alighting data, and train-by-train service information, none of which are present now.

Key assumptions and limits

- Interchange line shares are synthetic weights based on throughput and route length; they do not come from measured interchange transfers.
- Capacities, fleet sizes, headways, signaling minima, turnaround values, and line-specific policies are stored assumptions whose authoritative sources are not established in this repository.
- The waiting estimate assumes random passenger arrivals and regular headways. It does not read the timetable.
- Peak-hour logic uses whole-hour buckets, marking 17:00-20:59 as evening peak; this is coarser than the docstring's 20:30 ending.
- The modeled service window is 05:00 inclusive through 23:30 exclusive. The 05:00-23:00 demand chart is a daily profile, not the selected-time demand.

Relevant source: `app/scheduling_optimizer.py:38-306`; operations assumptions are authored in `pipeline/01_audit_and_clean_data.py:41-198` and emitted to `data/cleaned/metro_line_operations.csv`.

6. Network travel-time estimate and code map

How the graph works

Each graph vertex is `(station, line)`. Consecutive stations in the hand-authored line order connect in both directions. Edge time is estimated from Haversine distance at an assumed 32 km/h, floored at one minute; missing/repeated coordinates or an outlier segment use a 2.5-minute fallback. At an interchange, line-specific vertices are joined with a fixed five-minute transfer cost. Multi-source Dijkstra starts from all line vertices for the origin and returns the best time to each destination.

GRAPH OUTPUT IS A TIME ESTIMATE
The code returns one shortest time per reachable station. It does not reconstruct a step-by-step route, consult departure times, account for train waits/disruptions, or use station-specific transfer paths. The dashboard's displayed destination count is derived from this graph result; do not describe it as a verified official network count.

High-value files for a code walkthrough

File	What to explain
`run.py`, `app/__init__.py`	App entry, Flask object, import/WSGI lifecycle.
`app/routes.py`	Routes, input parse, API response, component orchestration.
`app/forecasting.py`	Model artifacts, feature schema, inference, interpolation, daily profile.
`pipeline/01_audit_and_clean_data.py`	Station cleanup, interchange labels, hand-authored service inputs.
`pipeline/02_generate_calibrated_dataset.py`	Scenario assumptions, line distribution, random generation.
`pipeline/03_train_and_evaluate_forecaster.py`	Feature engineering, chronological split, model and metric comparison.
`app/scheduling_optimizer.py`	Frequency bounds, capacity budget, risk labels, narrative.
`app/network_topology.py`, `app/travel_graph.py`	Ordered stations, aliases, edge weights, interchange cost, Dijkstra.
`app/static/js/dashboard.js`, `app/templates/index.html`	Browser controls, API request, chart/table rendering.

Technology snapshot

Python + Flask 3.1.3; pandas 3.0.5; NumPy 2.5.2; scikit-learn >=1.5.0; joblib >=1.4.0; Jinja2 3.1.6; vanilla browser JavaScript; Chart.js 4.4.1 via CDN; Gunicorn in `Procfile`. There is no database or frontend build system. No lockfile or pinned Python runtime is tracked, and joblib/scikit-learn version compatibility matters for model artifacts.

Versions above are declarations from `requirements.txt` or the template URL, not a verified production environment.

7. Limitations, risks, and improvement path

Classification	Finding
VERIFIED IMPLEMENTATION	Three-step synthetic data/model pipeline; saved-model inference; date-aware dashboard; rule-based service simulation; graph-based time estimates.
VERIFIED LIMITATION	No observed passenger counts, OD/transfer data, real-time state, database, automated tests, or verified current timetable in the repository.
POTENTIAL RISK	Unknown station/line values are not clearly rejected at the API boundary; joblib artifacts require trusted files and compatible versions; no auth/rate limit is shown.
NOT VERIFIED	Production Render configuration/health, official sources for capacities/headways, provenance/license of station seed file, runtime and memory measurements.
PROPOSED IMPROVEMENT	Bring in licensed ground truth and sourced operations inputs, evaluate on observed future periods, validate API inputs, add test coverage, pin runtime/dependencies, and version data/model together.

Prioritized next engineering steps

- **1. Establish trustworthy data:** source and license station topology, real schedules, capacities, and timestamped entries/exits; record retrieval dates and transformations.
- **2. Define the right target:** distinguish station entries/exits per hour, boardings per line, train onboard load, and capacity utilization instead of treating them as interchangeable.
- **3. Validate honestly:** hold out future observed dates and stations where appropriate; compare simple baselines; report error by station, time, and day type.
- **4. Protect reproducibility:** pin Python and artifact-sensitive packages; store data/model version metadata; document regeneration and deployment requirements.
- **5. Test boundaries:** service opening/closing minutes, holidays, unknown stations, feature schema, missing files, interchange sums, schedule limits, and disconnected stations.
- **6. Reduce operational ambiguity:** label the present outputs as modeled estimates and keep any live dispatch capability outside scope until approved data and safety review exist.

8. Project explanations for interviews

30 seconds

30 seconds
I built MetroInsights, a Flask-based Delhi Metro demand and service-planning simulation. It combines a synthetic-data-trained station forecast, an assumption-driven headway calculator, interchange line estimates, and topology-based travel-time estimates. The current version is a prototype: it is not trained or validated on observed station-level passenger counts.

1 minute

1 minute
MetroInsights lets a user choose a station, date, time, and line, then see a demand curve and a simulated service scenario. I created a three-step data pipeline: clean station metadata, generate two years of synthetic station-hour scenarios, then train and chronologically evaluate a HistGradientBoostingRegressor. The app loads the saved model, builds date/time/station features, estimates demand, allocates interchange demand using synthetic shares, and calculates headways under fleet and signaling assumptions. It also estimates destination times with a line-expanded graph and Dijkstra. The main limitation is that the targets and many operational values are assumptions, so the holdout metrics describe synthetic-data fit only.

2 minutes

2 minutes
The product is an interactive planning prototype, not a live DMRC operations system. The browser sends a station/date/time request to a Flask JSON endpoint. The backend derives date context, predicts hourly demand using a saved scikit-learn model and 23 features, and linearly interpolates the selected minute. At interchanges, it splits demand using a generated line-share file. A separate rule engine compares estimated boarding demand with a comfort-clearance budget, limits frequency using fleet/turnaround and signaling assumptions, and reports a headway, utilization, and headway-over-two wait estimate. The route module builds a graph whose nodes are station-line pairs, uses coordinate-based edge estimates and transfer penalties, and runs Dijkstra for destination times. Training uses 3.15 million generated records and a date-based holdout. The reported MAE/R-squared are against the synthetic generator, not real passengers. The most important next steps are obtaining observed demand and verified operations data, validating against future observed periods, and adding automated tests.

5 minutes

5 minutes

Start with the problem and the boundary. MetroInsights demonstrates how a planning user might compare estimated station demand with a possible service level. It is a prototype for exploring scenarios, not an operational control system. The repository does not contain observed station-by-hour AFC counts, train positions, a current timetable, or verified train-capacity and fleet constraints, so I describe its outputs as estimates and assumptions.

The data pipeline has three stages. Pipeline 01 cleans a static station file by trimming name markers, applying a small coordinate patch list, and adding line and interchange metadata from the project's hand-authored topology. It also writes an operations CSV from configurable constants. Pipeline 02 assigns project typologies and station weights, applies weekday/weekend hour profiles plus month and listed-occasion factors, then samples synthetic integer counts. The resulting scenario dataset covers 728 dates, 228 stations, and 19 hourly buckets, or about 3.15 million records. Pipeline 03 derives calendar, cyclical time, station, line, interchange, typology, and historical aggregate features. It holds out dates starting 2026-10-01, compares simple baselines and Ridge with HistGradientBoostingRegressor, and saves the estimator, inference metadata, and metrics report.

For the model, I would explain that sine/cosine encodings represent recurring time cycles, while historical averages provide station-specific context. The training code calculates prior-history features without using the current row's target, and uses training-period summaries for holdout inference. The deployed forecaster loads these saved artifacts rather than retraining on every request. The reported synthetic holdout MAE is 27.90 riders per hour and R-squared is 0.9947, but these describe agreement with generated targets. Because training and test targets are produced by the same assumptions, the score is not evidence of accuracy on real passenger counts.

At runtime, the browser submits a station, date, time, and optional line to Flask. The route parses the input, derives the date context, and checks the modeled 05:00 through 23:30 service window. The forecast module builds the model's 23-feature row, returns an hourly value, linearly interpolates a minute estimate, and produces a daily 05:00-23:00 curve. The frontend updates its charts, tables, and service KPIs. The 130 MiB generated CSV is ignored by Git and is not required to serve requests; it is required if the model is to be retrained, and the generator can recreate it.

The scheduling component is a transparent rule engine. It estimates boarding clearance from comfort capacity after an assumed through-load factor, compares hourly demand with baseline clearance capacity, bounds a candidate frequency using fleet/turnaround and signaling assumptions, rounds to practical headways, and calculates utilization and a headway-over-two wait estimate. Interchange demand is split with generated line weights. These are scenario calculations: the utilization is not train occupancy, and it does not know the people already onboard, exits, or boardings on a particular arriving train.

The travel-time component uses `(station, line)` graph nodes to preserve line identity. Adjacent stops connect, transfer edges add a fixed penalty, and multi-source Dijkstra finds the shortest estimated time per destination. It does not produce an itinerary or model departure times and disruptions.

I would close with the engineering next steps. Acquire licensed observed demand and traceable operations sources; define separate targets for entries, exits, transfer flow, and onboard train load; evaluate against future observed periods and station slices; add tests for time boundaries, inputs, artifacts, and graph coverage; and pin the runtime and dependency versions. That plan preserves the useful prototype while being honest about what must be verified before anyone relies on it operationally.

Introductory interview questions

What does MetroInsights do?

It explores simulated station demand and service-frequency scenarios through a Flask dashboard.

What data trained the model?

Generated station-hour scenarios. The repository does not provide observed station-level passenger counts.

Why use a time-based split?

The scenario is date-driven; training on earlier dates and evaluating on later dates better resembles future prediction than a random row split.

What does R-squared 0.9947 prove?

Only that the model fits this synthetic holdout well. It does not prove the same accuracy on real ridership.

Is capacity utilization actual train occupancy?

No. It is modeled hourly station demand divided by assumed boarding-clearance capacity.

What would you do next?

Acquire licensed ground truth and sourced operational data, validate on future observations, and add boundary/API/model tests.

A GOOD INTERVIEW HABIT

Name the evidence, state the assumption, and explain what would be needed to verify it. That makes the prototype's limitations part of a credible engineering discussion rather than something to hide.

Repository references: `MetroInsights_Study_Materials/00_Project_Analysis.md`; `DATA_AUDIT.md`; `read.md`; `requirements.txt`; `Procfile`; and the source files named in the sections above. Metrics are transcribed from `models/evaluation_metrics.json`. Repository statements about external feeds and provenance are not independently certified by this PDF.